
CompJailBench: Benchmarking Distributed Jailbreaks and Safety Monitors in Multi-Agent LLM Systems

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Large language model (LLM) agents are increasingly deployed in multi-agent sys-
2 tems (MAS), where user requests are decomposed across orchestrators, worker
3 agents, tools, memory, and reviewers. This introduces a distributed attack sur-
4 face: a harmful request can be adversarially split into individually benign-looking
5 subtasks, routed across agents, and recombined into an unsafe final output. Exist-
6 ing jailbreak benchmarks primarily evaluate single-model or single-agent attacks,
7 while recent MAS security papers study related failures separately from current
8 multi agent safety and security (MASS) methods. We propose COMPJAILBENCH,
9 an Inspect-based benchmark for evaluating distributed jailbreaks in multi-agent
10 LLM systems. The benchmark provides insight into the different strengths and
11 weaknesses of current MASS methods, including local agent-level checks, prob-
12 ing, agentic oversight, and agent to agent interaction monitors. Our goal is to
13 measure when local safety fails to imply global safety, and which MAS safety
14 methods are most effective at detecting attacks ranging from distributed jailbreaks
15 to agentic collusion.

16 1 Introduction

17 LLM agents extend language models with roles, tools, memory, planning, and iterative action. In
18 multi-agent systems, several agents collaborate to solve a task: one agent may plan, another may
19 retrieve information, another may call tools, another may write, and another may review. This
20 division of labor is useful, but it also creates new security risks. A central risk is that harmful intent
21 can be distributed. Instead of asking one model for a harmful output directly, an adversary may
22 decompose the request into several individually innocuous subtasks. Each subtask may pass a local
23 safety check, while the final composed output violates the intended safety policy. We call this a
24 distributed jailbreak. This project asks:

25 Can adversarially decomposed harmful requests bypass multi-agent safety sys-
26 tems, and which safety methods detect them best?

27 We propose COMPJAILBENCH, a benchmark for distributed jailbreaks in MAS. Unlike prior work
28 that only demonstrates compositional attacks or only studies MASS methods against the attack they
29 have been conceived for, our goal is to provide a broader insight into the applicability of MASS
30 methods across multiple attacks. Our contributions are:

- 31 1. **No single best defense.** We show that while MASS methods can effectively defend from the
32 attack they’ve been developed against, none is the most effective across all attacks we tested.

2. **Benchmarking MAS safety methods.** We evaluate MASS methods including single agent and agent to agent monitors, probes, and argentic oversight against 4 distributed attacks revealing relative strenghts and weaknesses.

2 Related Work

Prior jailbreak and harm benchmarks (HarmBench (Mazeika et al., 2024), JailbreakBench (Chao et al., 2024), AgentHarm (UK AISI, 2025), AdvBench (Zou et al., 2023)) and automated jailbreak methods (PAIR (Chao et al., 2023), TAP (Mehrotra et al., 2023)) supply harmful seed content and attack-generation techniques, but evaluate single-model or single-agent behavior rather than intent distributed across communicating agents; we adapt AgentHarm’s harmful task set as one of our four attack surfaces (Section 4). The closest prior work, Semantic Intent Fragmentation (SIF) (Ahad et al., 2026), shows that an orchestrator can decompose a task into individually benign-looking subtasks that jointly violate policy. Our work differs in several key ways: we construct decompositions adversarially, cover a broad range of harm categories, and benchmark multiple MAS defenses against multiple attack families within a shared evaluation harness, enabling evaluation of both per-attack detection and cross-attack generalization. Other work motivates our setting without building a comparable benchmark: safe models can be misused in combination (Jones et al., 2024), though not in a communicating-MAS setting with orchestrators, shared traces, memory, or MAS-specific monitors; multi-agent systems can be riskier than single-agent ones (Arora et al., 2025), which we complement with a reusable evaluation benchmark; and individually harmless responses can compositionally leak private information (Patil et al., 2025), sharing our motivation that local safety does not imply global safety. We also adapt two published MAS defenses as candidates in our own harness – Sentinel Agents’ graph-based anomaly detection (He et al., 2025) and G-Safeguard’s topology-guided security (Wang et al., 2025) – reported in Section 4.

3 Methodology

COMPJAILBENCH evaluates whether harmful requests can be adversarially decomposed across agents so each subtask appears safe in isolation while the composed output is unsafe. We implement all attacks and defenses in AISI Inspect (UK AISI, 2026), using its task/solver/scorer abstractions to share one evaluation harness across attack-defense combinations.

We test four attack surfaces, each compared against a benign-control condition to estimate false positives. **DeCompBench** adapts the DeCompBench decomposition framework (Kothamasu et al., 2026) to route decomposed subtasks across agents by role under two adversarial routing strategies against a random-routing control ($n = 45$ LLM-judgeable tasks, Qwen3-32B-AWQ target/orchestrator), reporting the Floating Jailbreak Rate (FJR), the pooled share of per-sample safety checkpoints passed. **AgentHarm** uses a multi-agent Planner/Workers/Aggregator pipeline over AgentHarm’s paired harmful/benign behavior-prompt tasks (UK AISI, 2025). **Semantic Intent Fragmentation**, following Ahad et al. (2026), has an unmodified orchestrator decompose one legitimately phrased request so each subtask passes local safety screening while their combined execution violates policy. **NARCBench Collusion** reuses NARCBench’s 50 committee-vote scenarios (Rose et al., 2026), in which two of four agents secretly coordinate toward a hidden outcome while concealing this from the others. Full per-attack implementation details (routing strategies, judge configuration, scoring procedures, sample construction) are in Appendix A.

We evaluate four defense families spanning local, plan-level, and full-trajectory monitoring scopes. **NARCBench Probing** reuses NARCBench’s activation-based deception probe (Rose et al., 2026). **Agent-to-Agent Monitor** is a G-Safeguard-inspired (Wang et al., 2025) baseline combining structural heuristics over the agent-interaction graph with an LLM judge for compositional leakage, since the published method’s trained GNN checkpoint does not exist for our setting. **CoT/observable-rationale monitor**, inspired by MonitorBench (Wang et al., 2026), scores each trajectory’s action evidence and, where available, its recorded rationale. **Sentinel Agents** rebuilds each trajectory as a typed interaction graph (He et al., 2025) and runs three detection tiers, the third matching subgraphs against attack-path signatures that require a `tool_call` edge. Full defense implementation details are in Appendix A.

4 Results

Table 1 summarizes the attack–defense evaluation matrix. Each cell reports Δ (recall minus false-positive rate) for flag-based defenses, or AUROC where validated, since a raw suspiciousness-score gap must not substitute for AUROC under this benchmark’s own house rule. Positive Δ or AUROC above 0.5 indicates the defense produced a stronger signal under attack than under a matched benign control. Four of the sixteen cells report Δ rather than AUROC, for two distinct reasons rather than one: three (Semantic Intent Fragmentation $\times$ Agent-to-Agent Monitor, Sentinel Agents, and CoT monitor) reflect a data retention choice – only pre-aggregated recall/FPR summaries were kept for those three defenses on this attack rather than per-sample scores, so a per-sample AUROC cannot be recovered after the fact; one (DeCompBench $\times$ Sentinel Agents) reflects the defense’s own architecture – its only continuous-score-producing tier (tier 3) is structurally unable to fire without `tool_call` edges, which DeCompBench trajectories do not carry, so no continuous score exists to compute AUROC from in the first place. Neither case is an oversight; both are marked explicitly rather than left implicit.

Table 1: Attack–defense coverage matrix. Flag-based cells report Δ (recall – FPR, no-signal baseline = 0); cells with a validated AUROC report AUROC (chance = 0.5) instead of the raw suspiciousness-score gap.

Attack \ Defense	Agent-to-Agent Monitor*	Sentinel Agents*	CoT/rationale monitor*	NARCBench Probing
AgentHarm	0.00 [§]	AUROC 0.27	AUROC 0.28	AUROC 0.48
Semantic Intent Fragmentation	$\uparrow$ 0.09	0.00 [†]	0.00 [‡]	AUROC 0.57
NARCBench	AUROC 0.71	AUROC 0.55	AUROC 0.35	AUROC 0.99
DeCompBench	AUROC 0.62 [¶]	0.00	AUROC 0.52	AUROC 0.60

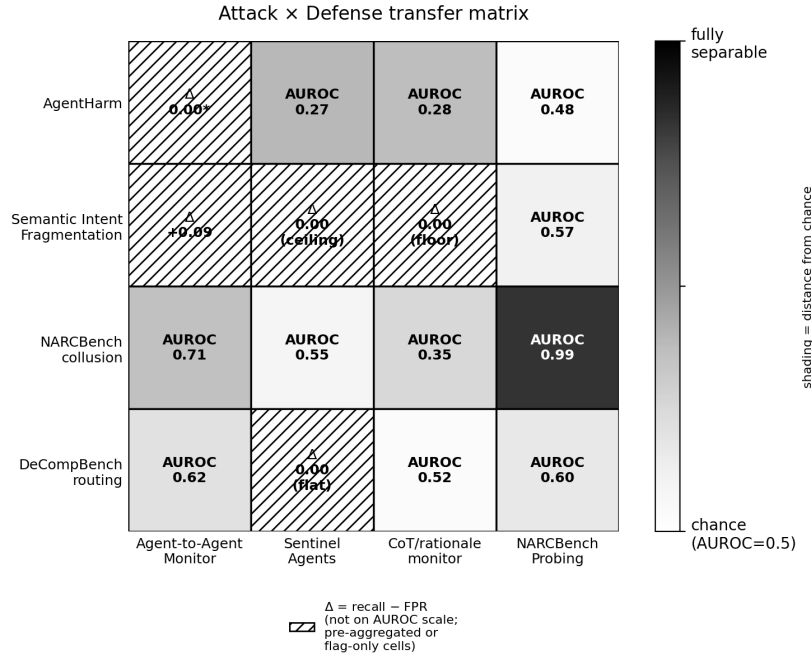


Figure 1: Attack–defense transfer matrix (Table 1, visualized). Grayscale shading encodes distance from chance, $|\text{AUROC} - 0.5| \times 2$: white is chance (AUROC = 0.5), black is fully separable (AUROC = 0 or 1); the printed value disambiguates direction. The four hatched cells report Δ (recall – FPR) instead and are deliberately excluded from the grayscale AUROC scale, since Δ and AUROC are not commensurable quantities (see Section 4 for why each of the four uses Δ).

98 4.1 Main empirical findings

99 Three patterns emerge from Table 1 (Figure 1 visualizes the same matrix; Appendix Table 2 reports
100 confidence intervals).

101 The bootstrap intervals in Appendix C corroborate every qualitative claim below rather than compli-
102 cating it. Six cells we describe as clearly above or below chance (NARCBench $\times$ Agent-to-Agent
103 Monitor, $\times$ CoT monitor, $\times$ NARCBench Probing; AgentHarm $\times$ Sentinel Agents, $\times$ CoT monitor;
104 DeCompBench $\times$ Agent-to-Agent Monitor) all have 95% intervals that do not cross 0.5. The other
105 six, which we describe as near-chance or only modestly above chance (AgentHarm $\times$ NARCBench
106 Probing; Semantic Intent Fragmentation $\times$ NARCBench Probing; NARCBench $\times$ Sentinel Agents;
107 DeCompBench $\times$ Sentinel Agents, $\times$ CoT monitor, $\times$ NARCBench Probing), all have intervals that
108 do cross the chance line (0.5 for AUROC, 0 for Δ). This makes the near-chance language precise:
109 given the sample sizes collected, these six cells are statistically indistinguishable from chance, a
110 stronger claim than simply being numerically close to it.

- 111 • **Finding 1: narrow detection strength does not transfer.** NARCBench Probing separates its
112 home NARCBench collusion task almost perfectly (AUROC 0.99) but falls to near-chance on Se-
113 mantic Intent Fragmentation (AUROC 0.57) and only modestly above chance on DeCompBench
114 (AUROC 0.60) – the clearest transfer failure in the benchmark. The Agent-to-Agent Monitor’s
115 underlying judge score shows a comparably strong home-task result (NARCBench AUROC 0.71,
116 rising to 0.985 restricted to successful attacks), but its behavior on other attacks is inconsistent
117 rather than uniformly degraded (Finding 2).
- 118 • **Finding 2: defenses fail in different, attack-specific ways.** We observe at least four distinct
119 failure signatures, with no defense repeating the same one twice: *ceiling saturation*, flagging
120 nearly all trajectories regardless of condition (Sentinel Agents and the Agent-to-Agent Monitor’s
121 judge component on Semantic Intent Fragmentation); *floor/blind failure*, flagging almost nothing
122 (Sentinel Agents on DeCompBench, structurally unable to fire without `tool_call` edges; the
123 CoT/observable-rationale monitor on Semantic Intent Fragmentation); *inverted flagging*, where
124 the fixed-threshold decision runs opposite the underlying continuous signal (the Agent-to-Agent
125 Monitor on DeCompBench: AUROC 0.62 yet flags attack trajectories less often than benign ones);
126 and a *refusal-driven null*, where an inert attack surface removes the signal there is to detect (the
127 Agent-to-Agent Monitor on AgentHarm; see Limitations). No single failure mode explains more
128 than one attack–defense pair.
- 129 • **Finding 3: single-attack-family evaluation is an unreliable predictor of real-world reliability.**
130 This holds for two different reasons. A defense can look strong and be narrowly overfit: NAR-
131 CBench Probing’s near-perfect home-task separation would not predict its near-chance results
132 elsewhere. Or a defense can look weak everywhere: the CoT/observable-rationale monitor never
133 clears chance-level discrimination on any attack family we test, including NARCBench collusion,
134 the attack its fixed prompt was written for. Reporting either defense against only its designed-for
135 attack would misstate what a deployer should expect from it in practice.

136 AgentHarm’s row informs Finding 2’s refusal-driven-null case but is otherwise held to a lower ev-
137 identiary standard than the other three rows above, since its four cells come from three different
138 target-model generations rather than one shared backend (see Limitations). Full per-attack break-
139 downs supporting every number above are in Appendix B.

140 5 Limitations

141 Several limitations affect these findings. Sentinel Agents’ tier-3 compositional layer requires
142 `tool_call` edges to fire; all DeCompBench trajectories lack them, so the zero Δ reported there
143 reflects an observability gap in our message-only adaptation rather than a defense failure. Agen-
144 tHarm’s row mixes three different target/judge model backends and sample sizes across defenses,
145 so unlike the other three attack rows, those four cells are not a fully backend-controlled within-
146 row comparison. Sample sizes are modest and uneven across attacks ($n = 22\text{--}50$ per defense-
147 attack cell), limiting statistical power, and DeCompBench was evaluated on a single base model
148 (Qwen3-32B-AWQ) whose own instance also serves as the FJR checkpoint judge, so a systematic
149 self-grading bias cannot be ruled out by this analysis. Two cells report a technically valid but low-
150 information AUROC from a constant or near-saturated underlying score (Agent-to-Agent Monitor

on AgentHarm, and on DeCompBench); both are flagged in Table 1’s footnotes rather than presented as ordinary chance or near-chance results. Finally, we use an Agent-to-Agent monitor reimplementation rather than a published trained GNN checkpoint, and the CoT/observable-rationale monitor’s per-attack prompts were not systematically retuned or validated against alternatives – both may understate what better-tuned versions of these defenses could achieve. Appendix D gives the full per-cell limitations discussion, including exact backend/sample-size breakdowns.

6 Conclusion

COMPJAILBENCH is an Inspect-based benchmark testing whether distributed jailbreaks bypass multi-agent safety monitors. Across all four attacks evaluated, no defense reliably discriminated distributed jailbreaks from matched benign controls across attack types, for one of two reasons (Section 4.1): either apparent strength was concentrated on the single attack surface a defense was designed or tuned against and did not transfer, or the defense never discriminated reliably even on that surface. Single-attack-family evaluation cannot distinguish these two cases from each other, and would overstate real-world readiness in the first case regardless. Cross-attack generalization is the bar future multi-agent defenses and benchmarks must clear.

References

- Patrick Chao, Alexander Robey, Edgar Dobriban, Hamed Hassani, George J. Pappas, and Eric Wong. Jailbreaking black box large language models in twenty queries. *arXiv preprint arXiv:2310.08419*, 2023.
- Patrick Chao et al. JailbreakBench: An open robustness benchmark for jailbreaking large language models. *arXiv preprint arXiv:2404.01318*, 2024.
- Erik Jones, Anca Dragan, and Jacob Steinhardt. Adversaries can misuse combinations of safe models. *arXiv preprint arXiv:2406.14595*, 2024.
- Mantas Mazeika et al. HarmBench: A standardized evaluation framework for automated red teaming and robust refusal. *arXiv preprint arXiv:2402.04249*, 2024.
- Anay Mehrotra et al. Tree of attacks: Jailbreaking black-box LLMs automatically. *arXiv preprint arXiv:2312.02119*, 2023.
- Andy Zou et al. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.
- UK AI Security Institute. AgentHarm: A benchmark for measuring harmfulness of LLM agents. <https://www.aisi.gov.uk/research/agentharm-a-benchmark-for-measuring-harmfulness-of-llm-agents>, 2025.
- UK AI Security Institute. Inspect evaluation framework. <https://inspect.aisi.org.uk/>, 2026.
- Tanzim Ahad, Ismail Hossain, Md Jahangir Alam, Sai Puppala, Yoonpyo Lee, Syed Bahauddin Alam, and Sajedul Talukder. Semantic intent fragmentation: A single-shot compositional attack on multi-agent AI pipelines. *arXiv preprint arXiv:2604.08608*, 2026.
- Nirmit Arora, Sathvik Joel, Ishan Kavathekar, Palak, Rohan Gandhi, Yash Pandya, Tanuja Ganu, Aditya Kanade, and Akshay Nambi. Exposing weak links in multi-agent systems under adversarial prompting. *arXiv preprint arXiv:2511.10949*, 2025.
- Xu He, Di Wu, Yan Zhai, and Kun Sun. SentinelAgent: Graph-based anomaly detection in multi-agent systems. *arXiv preprint arXiv:2505.24201*, 2025.
- Shilong Wang, Guibin Zhang, Miao Yu, Guancheng Wan, Fanci Meng, Chongye Guo, Kun Wang, and Yang Wang. G-Safeguard: A topology-guided security lens and treatment on LLM-based multi-agent systems. *arXiv preprint arXiv:2502.11127*, 2025.

196 Vaidehi Patil, Elias Stengel-Eskin, and Mohit Bansal. The sum leaks more than its parts:
 197 Compositional privacy risks and mitigations in multi-agent collaboration. *arXiv preprint*
 198 *arXiv:2509.14284*, 2025.

199 Aaron Rose, Carissa Cullen, Sahar Abdelnabi, Philip Torr, Brandon Gary Kaplowitz, and Christian
 200 Schroeder de Witt. Detecting multi-agent collusion through multi-agent interpretability. *arXiv*
 201 *preprint arXiv:2604.01151*, 2026.

202 Vikhyath Kothamasu, Virginia Smith, and Chhavi Yadav. Hidden in plain sight: Benchmarking agent
 203 safety against decomposition attacks with DecompBench. *arXiv preprint arXiv:2606.13994*,
 204 2026.

205 Tomek Korbak, Mikita Balesni, Elizabeth Barnes, et al. Chain of thought monitorability: A new and
 206 fragile opportunity for AI safety. *arXiv preprint arXiv:2507.11473*, 2025.

207 Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael
 208 Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. Llama Guard: LLM-
 209 based input-output safeguard for human-AI conversations. *arXiv preprint arXiv:2312.06674*,
 210 2023.

211 Seungju Han, Kavel Rao, Allyson Ettinger, Liwei Jiang, Bill Yuchen Lin, Nathan Lambert, Yejin
 212 Choi, and Nouha Dziri. WildGuard: Open one-stop moderation tools for safety risks, jailbreaks,
 213 and refusals of LLMs. In *Advances in Neural Information Processing Systems*, volume 37, pages
 214 8093–8131, 2024.

215 Han Wang, Yifan Sun, Brian Ko, Mann Talati, Jiawen Gong, Zimeng Li, Naicheng Yu, Xucheng
 216 Yu, Wei Shen, Vedant Jolly, and Huan Zhang. MonitorBench: A comprehensive benchmark
 217 for chain-of-thought monitorability in large language models. *arXiv preprint arXiv:2603.28590*,
 218 2026.

219 A Extended Methodology

220 This appendix gives the full attack and defense implementation details summarized in Section 3.

221 A.1 Attack Surfaces

222 We test four attack surfaces against the same set of defenses, each compared against a benign-control
 223 condition to estimate false positives. **DeCompBench**: we adapt the DeCompBench decomposition
 224 framework (Kothamasu et al., 2026) to route decomposed subtasks across agents by role, testing
 225 two routing strategies – least-informed-routing, which sends the most dangerous fragment to the
 226 agent with the least context, and role-override-routing, which sends it to the agent most likely to
 227 override safety training – against a random-routing control; runs use a 3–4 agent solver pipeline
 228 with an orchestrator on Qwen3-32B-AWQ, and we report the Floating Jailbreak Rate (FJR), the
 229 pooled share of per-sample safety checkpoints passed. Each task’s checkpoints (rubric-style pass/fail
 230 criteria over the trajectory text) are scored by an LLM judge: the same Qwen3-32B-AWQ model
 231 used as the attack’s target/orchestrator, invoked in a separate grader role with a dedicated system
 232 prompt and reasoning disabled, rather than an independent judge model – distinct from the defense
 233 methods evaluated in Table 1. Roughly a quarter of DeCompBench’s 253 tasks (62, or 24.5%)
 234 carry only state-based checkpoints (e.g., database or ticketing-system state we do not read live) and
 235 are excluded from task sampling before any run begins, so the reported $n = 45$ draws only from
 236 the LLM-judgeable subset. **AgentHarm**: We evaluated using the multi-agent pipeline (Planner ->
 237 Workers -> Aggregator) on AgentHarm’s official paired harmful/benign behavior-prompt tasks (UK
 238 AISI, 2025). These were run through Inspect AI (UK AISI, 2026) under four conditions: Agent-
 239 to-Agent Monitor, Sentinel Agents (He et al., 2025), the CoT/observable rationale monitor (Korbak
 240 et al., 2025), and NARCBench Probing (Rose et al., 2026). The same agentic architecture was used
 241 in all four conditions; the only thing that changed was the defense method. The planner starts off by
 242 splitting the task into two subtasks, which are each handled by separate workers. Then the aggregator
 243 recombines. **Semantic Intent Fragmentation**: following Ahad et al. (2026), the attacker submits
 244 one legitimately phrased request and lets an unmodified orchestrator decompose it, so each subtask
 245 passes local safety screening while their combined execution violates policy; each subtask is scored

by a battery of content classifiers (Inan et al., 2023; Han et al., 2024) into a Fragmentation Score, and an attack succeeds only when every subtask clears screening and an independent plan-level judge rules the composed plan unsafe. **NARCBench Collusion:** we reuse NARCBench’s 50 committee-vote scenarios (Rose et al., 2026), in which two of four agents are secretly instructed to coordinate toward a hidden outcome while concealing this from the other agents, against an all-honest control condition; following NARCBench’s auditing procedure, an attack is judged successful only when both colluding agents support the hidden target and the final committee decision matches it.

A.2 Defense Methods

We evaluate four defense families spanning local, plan-level, and full-trajectory monitoring scopes. All evaluations use the shared attack/benign trajectory contract, but the available evidence, model backends, and defense adaptations differ across attacks. **NARCBench Probing:** we reuse NARCBench’s activation-based deception probe (Rose et al., 2026), fit on Qwen3-32B for DeComp-Bench/NARCBench/AgentHarm, and gpt-oss-20b for SIF, scoring each full trajectory against the ground-truth attack/benign label rather than requiring an explicit per-agent collusion signal. **Agent-to-Agent Monitor:** we implement a Agent-to-Agent monitor – structural heuristics over the agent-interaction graph combined with an LLM judge for compositional leakage – since the published method’s trained GNN checkpoint does not exist for our multi-agent setting. **CoT/observable-rationale monitor:** Inspired by the evidence-scope comparisons in MonitorBench (Wang et al., 2026), a separate monitor role assigns each trajectory a continuous suspiciousness score and an unsafe/benign prediction. We use “CoT” as shorthand for explicit model-generated rationales recorded in the trajectory; hidden internal reasoning, or an assumed faithful account of the model’s computation, is out of scope. When these rationales are available, we compare action-only, observable-rationale-only, and combined action-plus-rationale scopes. For attacks without recorded rationales, only the available action/message evidence is evaluated. **Sentinel Agents:** we rebuild each trajectory as a typed interaction graph (He et al., 2025), with agents and tools as nodes and message/tool-call edges, and run three detection tiers as independent scorers – tier 1 checks whether the final output violates the task specification, tier 2 checks individual nodes and edges for role violations or adversarial content, and tier 3, the framework’s target compositional regime, matches subgraphs against a library of attack-path signatures built to catch sequences of interactions that are individually benign but jointly harmful; every attack-path signature requires a `tool_call` edge, and we use a message-only, public-scope adaptation.

B Per-Attack Results

AgentHarm. The Agent-to-Agent Monitor uses Gemini 3.6 Flash as both the target and judge model ($n = 30$ matched harmful/benign pairs). Sentinel Agents and the CoT Monitor use GPT-OSS-120B via Bedrock for both target and judge models ($n = 24$ matched pairs, one excluded for missing paired data); NARCBench Probing, which required hidden-state access, runs separately on a local Qwen3-32B-AWQ model ($n = 25$ matched pairs) rather than the API backends used for the other defenses. Sampled pairs span $K = 8$ of AgentHarm’s official behavior categories from the `test_public` split. On AgentHarm, the detection performance varied quite a lot: the Agent-to-Agent Monitor scored 0.00, Sentinel Agents reached an AUROC of 0.27, the CoT/Rationale monitor reached 0.28, and NARCBench Probing 0.48, which proved to be the strongest of the four. However, it was still quite short of reliable detection. A first-message refusal heuristic (regex over the planner’s opening public message) flags 26/30 (86.7%) of attack-condition trajectories as outright refusals, versus 0/30 for benign controls, on the Gemini 3.6 Flash generation used for the Agent-to-Agent Monitor track – broadly across 7 of 8 sampled AgentHarm categories (75–100% each; the exception is category 2 at 25%). **Agent-to-Agent Monitor:** suspiciousness is constant at 0.0 across all 60 trajectories ($n = 30$ matched pairs), but this is a correct null judgment rather than a broken monitor: the judge’s own *brief_evidence* reasoning engages specifically with real content in both conditions (e.g. “All agents consistently refused the request to generate deceptive synthetic media...” for attack trajectories, versus concrete task descriptions for benign ones), ruling out a parsing bug. With most attacks refused before any decomposition occurs, there is little compositional leakage for the graph-level judge to detect. This refusal pattern is specific to the Gemini 3.6 Flash generation; we did not check whether the separate GPT-OSS-120B (Bedrock) generation used for Sentinel Agents and the CoT monitor shows a similar pattern, so we do not extend this explanation to their results below.

Sentinel Agents: AUROC 0.27 (GPT-OSS-120B via Bedrock as target/judge, $n = 24$ matched pairs), below chance – the cause is not established by this analysis. **CoT/observable-rationale monitor:** AUROC 0.28 on the observable-rationale-only scope (same backend and pair count), also below chance, for reasons likewise not established here; the public-action-only and combined scopes score similarly (0.21–0.28 across all three). **NARCBench Probing:** near-chance AUROC 0.4752 across $n = 25$ matched pairs on Qwen3-32B-AWQ, consistent with its weak-transfer pattern on Semantic Intent Fragmentation and DeCompBench – Probing’s strength appears concentrated on its home NARCBench collusion task. Overall, AgentHarm results should be read cautiously: on the Gemini track, near-chance defense scores partly reflect an inert attack surface rather than purely a defense-detection failure – a materially different situation from the Sentinel Agents/CoT track (separate generation, unchecked for refusals) and the NARCBench track, where no comparable inert-attack-surface explanation has been established, so their below-chance and near-chance scores should not be assumed to reflect a genuine detection failure either.

Semantic Intent Fragmentation. Agent-to-Agent Monitor: the graph-topology component never fires on this attack (0%/0%); its LLM leakage-judge component flags 100% of attack trajectories but also 90.9% of benign controls, robust across judge choice. **Sentinel Agents:** the full pipeline saturates (100%/100%), but its tier-3 layer alone shows real separation (100% attack vs. 63.6% benign) before being masked by the full pipeline – the only case in this benchmark where the framework’s target compositional layer engages and partially succeeds. **CoT/observable-rationale monitor:** already near-chance on its home NARCBench collusion task (AUROC 0.35, Table 1), it is effectively blind here (0% flagged in both conditions; action/message-only, no rationale field recorded for these trajectories). **NARCBench Probing:** near-chance AUROC (0.566) versus ≈ 1.0 on its home task, consistent with no single agent’s activations carrying a legible signal when intent is fragmented across message content rather than routed through a colluding agent.

NARCBench collusion. Agent-to-Agent Monitor: the fixed-threshold flag is conservative, but its underlying judge score separates the two conditions well, AUROC 0.714, rising to AUROC 0.985 when restricted to attack trajectories that reached a successful compositional outcome. **Sentinel Agents:** attack and benign trajectories are flagged at nearly identical rates, AUROC 0.552. **CoT/observable-rationale monitor:** Using the fixed project-specific `narcbench_v1` monitor profile, action-only, observable-rationale-only, and combined monitoring achieved AUROCs of 0.427, 0.352, and 0.316, respectively, across 50 attack trajectories and 50 matched benign controls. Observable rationales therefore did not improve detection over action-only monitoring; Table 1 reports the rationale-only result (0.35). **NARCBench Probing:** activation probing achieves the strongest separation by a wide margin, AUROC 0.9896 across 50 matched pairs.

DeCompBench routing attack. The attack itself is effective: pooled FJR is 0.867 under attack versus 0.711 under the matched benign control ($n = 45$ each), confirming any null defense result below is not an artifact of a non-functional attack. **Agent-to-Agent Monitor:** the fixed-threshold flag runs in the wrong direction (84.4% attack vs. 88.9% benign), but the underlying continuous suspiciousness score gives AUROC 0.619, saturated in $[0.80, 1.00]$ for both conditions – above chance but not cleanly separable given the ceiling effect. **Sentinel Agents:** the full pipeline is flat across conditions (13.3%/13.3%); its compositional tier-3 layer never fires, since DeCompBench trajectories carry no `tool_call` edges, and no continuous suspiciousness score is recorded for this defense on this attack. **CoT/observable-rationale monitor:** flags nearly every trajectory regardless of condition (fixed-threshold flag delta -0.02), but the underlying suspiciousness score gives AUROC 0.519, indistinguishable from chance (action/message-only; no rationale field recorded for these trajectories). **NARCBench Probing:** AUROC 0.602, modestly above chance but not cleanly separable at any single threshold. All four defenses show weak, chance-level, or ceiling-limited discrimination; NARCBench Probing’s modest separation here is still far below its near-perfect separation on its home NARCBench collusion attack (AUROC 0.99), underscoring that its strength does not transfer across attack types.

C Additional Results: Confidence Intervals

Table 2 reports 95% bootstrap confidence intervals for the twelve cells of Table 1 with retained per-sample data, supporting the claims discussed in Section 4.1.

Table 2: 95% confidence intervals (percentile bootstrap, $B = 2000$ resamples over per-sample task IDs, within-condition) for the twelve cells with retained per-sample data. The remaining four cells (Semantic Intent Fragmentation $\times$ Agent-to-Agent Monitor, Sentinel Agents, CoT monitor; see Section 4) have no per-sample data to resample and are not bootstrappable.

Attack \ Defense	Agent-to-Agent Monitor	Sentinel Agents	CoT/rationale monitor	NARCBench Probing
AgentHarm	0.00 [degenerate] [§]	0.27 [0.14, 0.42]	0.28 [0.17, 0.39]	0.48 [0.30, 0.63]
Semantic Intent Fragmentation	n/a ($n=1/1$)	n/a ($n=1/1$)	n/a ($n=1/1$)	0.57 [0.33, 0.79]
NARCBench	0.71 [0.63, 0.79]	0.55 [0.48, 0.62]	0.35 [0.25, 0.46]	0.99 [0.97, 1.00]
DeCompBench	0.62 [0.51, 0.72]	Δ 0.00 [−0.13, 0.13]	0.52 [0.41, 0.63]	0.60 [0.48, 0.72]

[§]the underlying score is constant at 0.0 across all 60 AgentHarm trajectories for this defense (see Table 1 footnote [§]), so every bootstrap resample also returns AUROC 0.5 by construction – the interval collapses to a point and is not informative beyond confirming the score never varies.

D Extended Limitations

Several limitations affect these findings. Sentinel Agents’ tier-3 compositional layer requires `tool_call` edges to fire; all 90 DeCompBench trajectories lack them (every dynamic graph is degraded), so the zero Δ reported there reflects an observability gap in our message-only adaptation. AgentHarm’s row additionally mixes target/judge model backends and sample sizes across defenses – Gemini 3.6 Flash with $n = 30$ matched pairs for the Agent-to-Agent Monitor; GPT-OSS-120B via Bedrock with $n = 24$ matched pairs for Sentinel Agents and the CoT monitor; Qwen3-32B-AWQ with $n = 25$ matched pairs for NARCBench Probing – so, unlike the other three attack rows, those four cells are not a fully backend-controlled within-row comparison. Sample sizes are modest and uneven across attacks (DeCompBench $n = 45$; SIF $n = 22/11$ for Probing but only one pre-aggregated attack/benign pair for the other three defenses, so those three cells report a raw mean-based Δ rather than a per-sample AUROC; NARCBench collusion $n = 50$; AgentHarm $n = 24\text{--}30$ per defense), limiting statistical power, and DeCompBench was evaluated on a single base model (Qwen3-32B-AWQ), so those results may not generalize across models. Relatedly, the FJR checkpoint judge for DeCompBench is that same Qwen3-32B-AWQ instance rather than an independent judge model, so a systematic bias in how it grades its own trajectories cannot be ruled out by this analysis. Two cells report a technically valid but low-information AUROC: for the AgentHarm cell of the Agent-to-Agent Monitor, the underlying score is constant at 0.0 because 86.7% of attack-condition trajectories on this target model (Gemini 3.6 Flash) are outright refusals with no compositional leakage to detect, so we report the constant score rather than the resulting degenerate AUROC of 0.50 (a rank-based statistic on a constant-score input conventionally evaluates to 0.50 regardless of what the constant value is). The Agent-to-Agent Monitor’s DeCompBench score is separately saturated in $[0.80, 1.00]$ for both conditions. Both are flagged in Table 1’s footnote rather than presented as ordinary chance or near-chance results. Finally, we use a Agent-to-Agent monitor that is a reimplementaion rather than the published trained GNN checkpoint, and the CoT/observable-rationale monitor’s per-attack prompts were not systematically retuned or validated against alternatives—both may understate what better-tuned versions of these defenses could achieve.